LUT core-module ablation: three generations of lookup math

Comparison table, version 3

2026-09-11

Protocol (fixed for the whole paper set)

Geometry (the `exp_g_0193` geometry): LUT transformer $E = 384$, 6 layers, 6 attention heads; `CompressionMultiHeadLUT` with $H = 4$ **heads,** $tph = 128$ **tables per head**, $d_{in} = d_{out} = 48$, $n = 8$ **anchor pairs per table** (nap, $K = 2^n = 256$ cells). **Training and eval:** 16,000 steps, batch 48 rows × 512 tokens (device batch 12 × 4 gradient accumulation in the runs), untied embeddings, `random_seed` 1 / `lut_base_seed` 1000, LUT tables without weight decay, corrected eval (`evaluate_bpb_fixed`: 48 rows × 100 steps with the leading 12 rows skipped). Target hardware nebius-h100.

Geometry change from v2 ($H = 8$, $tph = 64$). $P = H \cdot tph = 512$ is unchanged, so **every core-LUT FLOP/MAC expression and every number in Table B is identical under either geometry.** What the geometry does change is outside the core: the `compress`/`decompress` projections (384→192→384 instead of 384→384→384, see the counting convention) and the output rank, which is capped at $H \cdot d_{out} = 192$ at $H = 4$.

Validation bpb. Three cells are filled from runs at exactly this protocol: 2.1 (`exp_n_0121`), 3.1 (`exp_g_0193`) and 3.2 (`exp_g_0195`); every other cell is TBD. Earlier numbers not measured on this protocol are not carried over: the Gen-2 results 1.2281 (`exp_g_0014`) and 1.2176 (`exp_n_0044`) ran at $n = 6$ with the old batch-coupled eval and tied embeddings, and `exp_g_0207`/`exp_g_0206` are $H = 8$, $tph = 64$, 48K-step runs.

Canonical $n = 2$ read-out temperature (a stated choice, reversible): τ learnable, initialised 0.5 (`exp_g_0195`). τ is a free hyperparameter of the $n = 2$ read-out, not part of the LightMHL math: the blend weight $\sigma(-2|u_{j^\star}|/\tau)$ is defined for any τ. The alternative `read_tau='auto'` imports margin statistics measured on a *different* run (`exp_g_0193`) and so ties row 3.2 to it. And the learnable setting is better by 0.0216 bpb. *Sensitivity:* `exp_g_0194` (`read_tau='auto'`, i.e. frozen at the measured per-layer Δm 0.033–0.108) reaches 1.182259. The frozen measured Δm is too sharp: `exp_g_0194` is +0.0094 *worse* than $n = 1$, while `exp_g_0195` is −0.0122 better. The learned τ ended at 0.158 / 0.292 / 0.334 / 0.380 / 0.465 / 0.669 by layer, 4–6× above the measured Δm and still drifting slightly at 16K. *Caveat:* the `exp_g_0194`-vs-`exp_g_0195` comparison changes two things at once, the init value *and* learnability, and cannot separate them: there is no frozen-0.5 run and no learnable-from-Δm run. Host and code differences do not explain the gap: the same-seed cross-host pair `exp_g_0191` (gpustar) vs `exp_n_0192` (nebius), identical config, agrees to a median 1.5e-4 and at most 5.3e-4 at matched steps 4K–14K (21 shared eval steps; the max occurs at step 4,500). That is roughly two orders of magnitude below the 0.0216 τ gap, so it explains essentially none of it.

Seed spread: provenance of the figures on record. 0.00335 is the gap between two *vanilla dense* seeds at 16K (`exp_n_0135` seed 1 = 1.165147 vs `exp_n_0176` seed 2 = 1.161798). It is a two-point range, not a standard deviation, and it was **not** measured on an LUT or at this geometry. `SWEEP_RESULTS.md`: the budget-law fit over "19 CompressionMHL runs at 16k" has residual sd ≤ 0.0035, an upper bound on 16K seed noise. The 4K proxy has a three-seed sd of 0.009642 (identical S5 config), which is a **lower bound**, because `random_seed` re-draws only 26.8% of the parameters. The same-seed cross-host repro pair above agrees to a median 1.5e-4 (max 5.3e-4). There are **zero seed replicates at this geometry** (see the remaining-work section).

Setup and notation

Shared model. LUT transformer, $E = 384$, 6 layers, 6 attention heads. The FFN is `CompressionMultiHeadLUT` with $H = 4$ heads, $tph = 128$ tables per head, $d_{in} = d_{out} = 48$: `compress` $E \to Hd_{in}$, the *core LUT* on $[T, H, d_{in}] \to [T, H, d_{out}]$, `decompress` $Hd_{out} \to E$. Only the core LUT differs between rows. Per head h, the core reads $z_h \in \mathbb{R}^{d_{in}}$.

Per table t (of the tph tables of head h): n fixed anchor pairs $(a_j, b_j) \subset [0, d_{in})$, $u_j = z_h[a_j] - z_h[b_j]$, $m_j = |u_j|$, bits $\beta_j = [u_j > 0]$, cell $c = \text{pack}(\beta) \in [0, K)$, table $W_t \in \mathbb{R}^{K \times d_{out}}$. $c' = c^{(j^\star)}$ is c with the bit of the least-confident pair $j^\star = \arg\min_j |u_j|$ flipped. The head output is $y_h = \sum_{t \in h}(\cdot)$. In the backward, $g = \partial L / \partial y_h$, $G_t(\ell) = \langle g, W_t[\ell] \rangle$, and any $\partial L / \partial u_j$ is scattered as $+$ to $z_h[a_j]$ and $-$ to $z_h[b_j]$. σ is the logistic sigmoid. The cell index is integer-valued: no gradient ever flows through it (no straight-through estimator in any row). $P = H \cdot tph = 512$ tables per token and $d = d_{out}$.

Counting convention for FLOPs and MACs

- **Scope:** the core LUT module only, **per token**, one layer. Multiply by T for a sequence and by 6 for the model. The `compress`/`decompress` projections are identical in every row and excluded: at $H = 4$ (384→192 and 192→384) they add $E \cdot H d_{in} + H d_{out} \cdot E = 73{,}728 + 73{,}728 = 147{,}456$ MACs ($\approx 294{,}912$ FLOPs) forward and about twice that backward per token per layer (`exp_n_0121`'s `corrected_score.json` lists `projection_flops_total: 147456`). That is *more* than the forward of every core variant below: 294,912 FLOPs > the 134,656 maximum core forward FLOPs, and 147,456 MACs > the 49,152 maximum core forward MACs.
- **FLOP** = one scalar floating-point operation: $+, -, \times, \div$, a float comparison, $|\cdot|$, exp, $\log \sigma$, σ each count 1. **MAC** = a multiply whose result is accumulated (weight × value, then added): **1 MAC = 2 FLOPs**, and MACs are included in the FLOP column. A pure accumulate (adding a looked-up row) is 1 FLOP and not a MAC.
- **Not counted:** memory reads (table-row gathers, anchor gathers), integer work (bit packing, XOR, index offsets), optimizer updates, and saving activations.
- **Forward** = inference cost (what an evaluated model executes). **Backward** = all *additional* arithmetic a training step needs, including training-only forward work (for example the alternative search in 1.1).
- Terms proportional to d, K and nK are exact. **Scalar per-table overheads** ($O(n)$ and constants) are counted from the code but approximate to within a few operations per table, since autograd's internal breakdown varies. d_{in} does not enter the core cost: it only sets the anchor index range.
- In the Gen-2 and Gen-3 cost expressions, k is the number of table cells whose rows enter the computation (for example $k = 2$ for a two-cell blend). Gen-1 expressions are written with their fixed two alternatives substituted. Temperature gradients ($O(1)$ per layer) are ignored. Gen-2 rows assume the confidence gate is *off*, as in the Gen-2 runs.

Gen-3 confidence forms

LightMHL multiplies each addressed row by a score computed from the margins. The score is computed at its call site `light_multi_head_lut.py:549-550`; it is consumed by `_bagged_sum` at `:565` for $n = 1$, and by `_blend_bag` at `:557-561` (as score $\cdot\, w$, `:677`) for $n = 2$. The function is `_confidence_score`, imported at `light_multi_head_lut.py:38` and defined at `fast_multi_head_lut.py:171-185`[1]. Three forms exist (g = `confidence_gain`, default 1):

form	score	score at a boundary / before[2]	configs using it (of 55 Gen-3)
`bounded`	$g \prod_j \sigma(2m_j)$	0.63	0
`bounded_norm`	$g\big(\prod_j \sigma(2m_j)\big)^{1/n}$ (geometric mean)	0.94	8
`margin`	$g\big(\sum_j m_j\big) \prod_j \sigma(2m_j)$ (LookupFFN kernel)	0.56	47 (including every run cited here[3])
`min_margin` (*proposed*, row 3.3)	$g\big(\min_j m_j\big) \prod_j \sigma(2m_j)$	0	not implemented

[1] `_confidence_score` is a shared helper used by FastMHL, LightMHL and BH4 (`bh4_multi_head_lut.py:56, 213`). It lives in the fast module for historical reasons and is arguably misplaced there; a common module would be the natural home.
[2] Median ratio of the score with one margin set to 0 to the score before, on random nap-8 margins.
[3] `exp_g_0193`, `exp_g_0194`, `exp_g_0195`, `exp_g_0206`, `exp_g_0207`.

None of the three existing forms vanishes at a cell boundary, so all of them leave the $n = 1$ forward discontinuous; only `min_margin` reaches 0 there. For $n = 1$ inference on CUDA without grad, the score is computed by a separate native kernel[4].

Table A — forward and backward math

Variant	Forward pass (math)	Backward pass (math)
GEN 1 — Spiking Manifesto basic model, rational uncertainty $U(u) = 0.5/(1+\lvert u\rvert)$[a] — both rows use exactly two alternatives: c and c'		
1.1 Hard forward, two-alternatives backward	$y_h = \sum_t W_t[c_t]$. Training only: the alternative is selected by $j^\star = \arg\min_j \lvert u_j\rvert$.	Weights: $\partial L/\partial W_t[c_t]$ += g (addressed row only). Input (surrogate — the derivative of 1.2's blend): $\frac{\partial L}{\partial u_{j^\star}} = -U'(u_{j^\star})\left(G_t(c_t) - G_t(c'_t)\right)$, with $-U'(u) = \frac{0.5\,\operatorname{sgn} u}{(1+\lvert u\rvert)^2}$; $\partial L/\partial u_j = 0$ for $j \neq j^\star$.
1.2 Soft forward, two-alternatives backward	$y_h = \sum_t \left[(1-U_t)\,W_t[c_t] + U_t\,W_t[c'_t]\right]$, $U_t = U(u_{t,j^\star})$. Continuous at $u_{j^\star} = 0$ (both weights $\frac{1}{2}$). Applied at eval too.	Weights: $\partial L/\partial W_t[c_t]$ += $(1-U_t)\,g$, $\partial L/\partial W_t[c'_t]$ += $U_t\,g$. Input: $\partial L/\partial u_{j^\star} = -U'(u_{j^\star})\left(G_t(c_t) - G_t(c'_t)\right)$ as in 1.1, which here is the *exact* gradient of the blend with both cells held fixed.
GEN 2 — FastMHL math[b]. Surrogate: $\rho_j = \frac{\lvert u_j\rvert}{T_s + \lvert u_j\rvert}$, $s_\ell = \sum_j \rho_j - 2\sum_{j:\,\mathrm{bit}_j(\ell)\neq\beta_j}\rho_j$, $\pi_\ell = \mathrm{softmax}_\ell(s_\ell/T_{sel})$ (signs β pinned; T_s, T_{sel} are temperatures)		
2.1 Hard forward, all-alternatives backward (all K cells)	$y_h = \sum_t W_t[c_t]$. ($c_t = \arg\max_\ell s_\ell$; no K-sized tensor is built.)	Weights: $\partial L/\partial W_t[c_t]$ += g. Input via the K-cell surrogate $\hat{y}_t = \sum_\ell \pi_\ell W_t[\ell]$ (never used in the forward): $\frac{\partial L}{\partial u_j} = \frac{1}{T_{sel}}\sum_\ell \pi_\ell\left(G_t(\ell) - \bar{G}_t\right)\frac{\partial s_\ell}{\partial \rho_j}\cdot\frac{T_s\,\operatorname{sgn} u_j}{(T_s+\lvert u_j\rvert)^2}$, $\bar{G}_t = \sum_\ell \pi_\ell G_t(\ell)$, $\partial s_\ell/\partial\rho_j = \pm 1$ (agree/disagree with β_j). All n pairs get gradient; reads all K rows.
2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward	$y_h = \sum_t \left[(1-w_t)W_t[c_t] + w_t W_t[c'_t]\right]$, $w_t = \sigma(-\Delta_t/T_{sel})$, $\Delta_t = 2\rho_{t,j^\star}$ (the exact 2-way softmax of s over $\{c, c'\}$). Applied at eval too.	Weights: $(1-w_t)\,g$ into $W_t[c_t]$ and $w_t\,g$ into $W_t[c'_t]$. Input: the K-cell surrogate of 2.1 (not the gradient of the forward that actually ran).
2.3 Hard forward, 2-alternatives backward	$y_h = \sum_t W_t[c_t]$	**TBD** — not implemented; open decision. `FastMultiHeadLut` raises `ValueError` for `backward_topk>0` unless `forward_mode='hybrid_smooth'` (`fast_multi_head_lut.py:1305`).
2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward	As 2.2.	Weights: as 2.2. Input (`backward_topk=1`): the exact gradient of the 2-cell blend with cells fixed. Only $j^\star$ gets gradient: $\frac{\partial L}{\partial u_{j^\star}} = -\frac{w_t(1-w_t)}{T_{sel}}\left(G_t(c'_t) - G_t(c_t)\right)\frac{2T_s\,\operatorname{sgn} u_{j^\star}}{(T_s+\lvert u_{j^\star}\rvert)^2}$.
GEN 3 — LightMHL math (LookupFFN-inspired). Score s_t per the confidence-form table above; the cell index uses detached u[c]		
3.1 Soft forward n=1, margin confidence, single-alternative backward	$y_h = \sum_t s_t\,W_t[c_t]$, $s_t = \left(\sum_j \lvert u_{t,j}\rvert\right)\prod_j \sigma(2\lvert u_{t,j}\rvert)$. **Piecewise-continuous:** the margin score stays positive at a sign flip, so y jumps by $s_t\left(W_t[c'] - W_t[c]\right)$; a min-margin score would make it continuous (row 3.3)[d].	Weights: $\partial L/\partial W_t[c_t]$ += $s_t\,g$. Input (only through the score): $\frac{\partial L}{\partial u_j} = G_t(c_t)\left(\prod_i \sigma(2\lvert u_i\rvert) + 2s_t\,\sigma(-2\lvert u_j\rvert)\right)\operatorname{sgn} u_j$ for all j. $\partial L/\partial W_t[c']$ is **exactly zero** (verified by autograd), and perturbing $W_t[c']$ leaves the input gradient unchanged: neither forward nor backward carries any signal about the neighbouring cell. At $n = 2$ both do.

continued on next page

[4] `native/lutorch/lutorch.cu:241-247`, reached via `light_multi_head_lut.py:700-707` (`_fused_eval`). $n = 1$ eval and $n = 1$ training therefore compute the score through different code, which agrees to ≈2e-7 in fp32. The bpb numbers come from the eval path. Gen 2 never uses this scored kernel.

Variant	Forward pass (math)	Backward pass (math)
3.2 Soft forward n=2, margin confidence, two-alternatives backward	$y_h = \sum_t s_t \left[\omega_0 W_t[c_t] + \omega_1 W_t[c'_t]\right]$, $\omega_1 = \sigma(-2\lvert u_{j^\star}\rvert/\tau)$, $\omega_0 = 1-\omega_1$, $\tau = e^{\log\tau}$ (learnable, initialised 0.5; see Protocol). Continuous at bit flips (the two cells swap roles at weights $\frac{1}{2}$), but it retains a smaller discontinuity where the two smallest margins swap and the second cell changes: about 8× smaller than 3.1's jump on trained weights[d].	Weights: $s_t\omega_0\, g$ and $s_t\omega_1\, g$ into the two read cells. Input: $\dfrac{\partial L}{\partial u_j} = (\omega_0 G_0 + \omega_1 G_1)\dfrac{\partial s_t}{\partial\lvert u_j\rvert}\operatorname{sgn} u_j + [j=j^\star]\, s_t \frac{2}{\tau}\omega_0\omega_1(G_0-G_1)\operatorname{sgn} u_j$, with $G_0 = G_t(c_t)$, $G_1 = G_t(c'_t)$. $\partial L/\partial \log\tau = \sum_t s_t\omega_0\omega_1(G_1-G_0)\, 2\lvert u_{j^\star}\rvert/\tau$.
3.3 Soft forward n=1, min-margin confidence, single-alternative backward (*proposed*)	$y_h = \sum_t \tilde{s}_t\, W_t[c_t]$, $\tilde{s}_t = g\left(\min_j \lvert u_{t,j}\rvert\right) P_t$, $P_t = \prod_j \sigma(2\lvert u_{t,j}\rvert)$, $g \approx 38$ (scale-matched to margin[e]). At a boundary ($\lvert u_i\rvert \to 0$, i becomes the argmin): $\tilde{s}_t = \frac{g}{2} C_i \lvert u_i\rvert + O(u_i^2)$, $C_i = \prod_{j\neq i}\sigma(2\lvert u_j\rvert)$, i.e. it vanishes linearly. So y is **continuous everywhere** (C^0, Lipschitz, piecewise smooth) but **not differentiable** at a boundary: the one-sided slopes are $-\frac{g}{2}C_i W_t[c]$ and $+\frac{g}{2}C_i W_t[c']$. It also has kinks where the argmin switches.	Weights: $\partial L/\partial W_t[c_t]$ += $\tilde{s}_t\, g$. Input (only through the score): $\dfrac{\partial L}{\partial u_j} = G_t(c_t)\Big(g\, P_t\, [j=j^\star] + 2\tilde{s}_t\,\sigma(-2\lvert u_j\rvert)\Big)\operatorname{sgn} u_j$, i.e. the gP term goes to the argmin anchor only. Ties use the first-index convention of `torch.min` (not `amin`, whose even split breaks the parity between Fast's analytic backward and Light's autograd). **TBD** — not yet implemented. Row 3.4 ($n=2$ + min-margin) is **deferred**: $n=2$ is already continuous at bit flips and min-margin does not remove its argmin-switch jump, so it would not test continuity. Run it only if 3.3 beats 3.1.

Notes to Table A.

[a] Other uncertainty functions may be tried later. The code also offers `UncertaintyMode.INVERSE_QUADRATIC`, $0.5/(1+u^2)$; see also the prior-evidence appendix on confidence forms.

[b] **Config trap (Gen 2 only):** `backward_topk` counts cells *beyond* the addressed one. Our "2 alternatives" is `backward_topk=1`; `backward_topk=2` would mean 3 cells. "All alternatives" is all $K = 2^n$ cells (`backward_topk=0`, a full softmax surrogate), not the n single-bit flips (that would be `backward_topk=n`). `backward_topk` never changes the weight gradient.

[c] A hard-forward Gen-3 variant still needs to be designed: nothing in the code trains LightMHL with a hard forward.

[d] **Measured** (`continuity_probe.py`, `continuity_probe_trained.py`). Two independent halves: **(i) Random init, float64, 300 draws — re-measured at the v3 geometry $H=4$,** $tph = 128$; the previous-geometry v2 values at $H=8$, $tph=64$ are in brackets. **3.1:** at the boundary the score is a median 0.514× the typical table score (p10 0.19, p90 1.27) [0.50×], never 0. The jump is a median 4.11% of $\|y_h\|$ (p90 9.55%, max 33.8%) [median 5.4%, p90 14%], against 4.59e-10 for a same-size step with no flip [4.6e-10], so it remains about eight orders of magnitude above the float floor: the discontinuity stands, just smaller. The shrink (median ratio 0.76, p90 ratio 0.68) is close to the $\approx 1/\sqrt{2} = 0.707$ expected because each head now sums twice as many tables: the expected effect, not an anomaly. **3.2:** the bit-flip jump is 6.6e-10, i.e. continuous; the argmin-switch jump is a median 1.08% (p90 3.1%) [median 1.85%]. The min-margin counterfactual gives 1.5e-9 [1.6e-9]. $\partial L/\partial W_t[c']$ at $n=1$ is exactly zero, with zero change in the input gradient. The native fp32 kernel agrees with torch to a median 2.0e-7 (max 3.5e-7) and shows the same jump (median 3.98%, slightly below the float64 torch figure because fp32 uses $\varepsilon = 10^{-4}$). **(ii) Trained — unchanged, and already at $H=4$,** $tph = 128$ (`exp_g_0193`/`exp_g_0194`, real val tokens): **3.1:** median 3.5% of the head output and 0.9% of the FFN output after `decompress`; about 1.2% of real margins lie within 10^{-2} of zero. **3.2:** the argmin-switch jump is a median 0.44% of the head output and 0.11% of the FFN output, about 8× smaller.

[e] The gain matches `margin`'s mean on the same margins; the ratio is stable through training (38.9 at init, 36.3 after 4K steps, 37.4 on `exp_g_0193`). These were already measured on $H=4$ runs (`exp_g_0193` and the `sweep_s05` cache), so the note is unchanged under the v3 geometry. **Confound:** `min_margin` changes continuity *and* selectivity together (within-token CV 1.7–2.0 vs 0.87–0.98 for `margin`; 10–27% of (token, table) scores below 10^{-3}), so a single 3.3-vs-3.1 arm cannot separate the two effects. Details: `notes_min_margin.md`.

Table B — cost and results

Per token, one layer, core LUT only. Concrete values at the v3 geometry $H = 4$, $tph = 128$, $d_{in} = d_{out} = 48$, $n = 8$ ($K = 256$), $P = 512$, $d = d_{out}$; every expression depends only on P, n and d, so the numbers are identical at $H = 8$, $tph = 64$. Validation bpb on the protocol above.

Variant	Forward FLOPs	Backward FLOPs	Forward MACs	Backward MACs	Validation bpb	Implementation (class used)
1.1	$P(2n + d)$ = **32,768**	$P(5d + 2n + 9)$ = **135,680**	0	$P \cdot 2d$ = **49,152**	**TBD** — not yet implemented in the FFN harness.	`spiky.lutorch.multi_head_lut.MultiHeadLut` (`smooth_mode=False`, `n_alternatives=1`, `uncertainty_mode=INVERSE_L1`); also `lut_fused.LUTLayer.LUTLayerBasic`. **Not wired into** `CompressionMultiHeadLUT`.
1.2	$P(4d + 4n + 4)$ = **116,736**	$P(8d + 9)$ = **201,216**	$P \cdot 2d$ = **49,152**	$P \cdot 4d$ = **98,304**	**TBD** — not yet implemented in the FFN harness.	`MultiHeadLut` (`smooth_mode=True`, `n_alternatives=1`). Not wired into `CompressionMultiHeadLUT`.
2.1	$P(2n + d)$ = **32,768**	$P(2Kd + 4nK + 9K + 11n + d)$ = **18,026,496**[f]	0	$P(Kd + 2nK)$ = **8,388,608**	**1.180622** (`exp_n_0121_ffnsw_nap8_tph128_16k`)[h]	`CompressionMultiHeadLUT(lut_impl='fast')` → `FastMultiHeadLut(forward_mode='hard', backward_topk=0)`; keys `lut_forward_mode`, `lut_backward_topk`.
2.2	$P(2kd + 4n + 5)$ = **117,248**	$P(2Kd+4nK+9K+11n+2kd)$ = **18,100,224**[f]	$P\,kd$ = **49,152**	$P(Kd + 2nK + kd)$ = **8,437,760**	**TBD** — not yet run	`FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0)`.
2.3	$P(2n + d) = 32{,}768$	n/a	0	n/a	**TBD** — not implemented; open decision	Would be `FastMultiHeadLut(forward_mode='hard', backward_topk=1)`, which the code rejects.
2.4	$P(2kd + 4n + 5)$ = **117,248**	$P(4kd + 9n + 15)$ = **241,152**	$P\,kd$ = **49,152**	$P\,2kd$ = **98,304**	**TBD** — not yet run	`FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1)` (note b).
3.1	$P(2kd + 7n)$ = **77,824**	$P(4kd + 8n)$ = **131,072**	$P\,kd$ = **24,576**	$P\,2kd$ = **49,152**	**1.172852** (`exp_g_0193`, in-run corrected eval)[i]	`CompressionMultiHeadLUT(lut_impl='light')` → `LightMultiHeadLUT(confidence_form='margin', read_top_n=1)`.
3.2	$P(2kd + 8n + 7)$ = **134,656**	$P(4kd + 8n + 11)$ = **235,008**	$P\,kd$ = **49,152**	$P\,2kd$ = **98,304**	**1.160637** (`exp_g_0195`, τ learnable, init 0.5; sensitivity: `exp_g_0194` 1.182259, see Protocol)	`LightMultiHeadLUT(confidence_form='margin', read_top_n=2, read_tau=0.5, learnable)`.
3.3	as 3.1 = **77,824**[g]	as 3.1 = **131,072**[g]	**24,576**	**49,152**	**TBD** — not yet implemented; open decision	Would be `LightMultiHeadLUT(confidence_form='min_margin', confidence_gain≈38, read_top_n=1)`; touchpoints in `notes_min_margin.md`.

Notes to Table B.

[f] Dominated by $2Kd$: the all-alternatives backward reads all K rows of every table. Memory: at $n = 8$ it holds several `[tokens, 512, 256]` fp32 buffers, about 12.9 GiB each at 24,576 tokens, so batch 48 needs micro-batching or gradient accumulation.

[g] Listed as identical to 3.1. A minimum over n margins costs the same $n - 1$ operations as a sum over n. In the backward the gP term lands on one anchor instead of n, which saves $P(n - 1) = 3{,}584$ FLOPs, but that is smaller than the stated scalar-overhead tolerance, so it is not claimed as a difference.

[h] This value comes from a **re-score of the saved run under the corrected eval** (0 missing keys), not from an in-run eval; the original batch-coupled number was 1.191455. The run has `lut_learnable_temps: true` (+12 parameters vs `exp_g_0193`), and `lut_backward_topk` is absent, i.e. 0 = all 2^n cells.

[i] `exp_g_0193`'s `lut_learnable_temps`, `lut_forward_mode` and `lut_forward_confidence` keys are inert on LightMHL.

Open decisions

1. **Row 2.3** (hard forward, 2-alternatives backward) is rejected by `FastMultiHeadLut` (`ValueError` unless `forward_mode='hybrid_smooth'`). Either patch the code to make it legal, or drop the row.
2. **Row 3.3** (min-margin): implement it now, or keep it planned but not run. The spec, scale analysis and touchpoints are in `notes_min_margin.md`.
3. Rows 1.1/1.2 cannot run at all until Gen 1 is wired into `CompressionMultiHeadLUT`: `MultiHeadLut` takes a shared `[B, input_dim]` input rather than the per-head `[T,H,d_in]` layout (for example via `anchor_candidates` restricting each head's anchors; see the remaining-work section for the shape).

Where the code disagrees with the original brief

1. **Gen 1 is not wired into the experiment harness.** `CompressionMultiHeadLUT` accepts only `lut_impl` $\in$ {`fast`, `light`, `bh4`} (`compression_mhl.py:184`), and no `model_build.py` key reaches `MultiHeadLut`.
2. **In 1.1 the input gradient is a surrogate.** It is the derivative of 1.2's soft blend, while the weights update only the addressed row.
3. **The paper's status report writes** $u(\delta) = 0.5/(T + |\delta|)$ with a per-LUT temperature. The code's $U(u) = 0.5/(1 + |u|)$ has none (`l_projection.py:96`, `lutorch.cu:797`).
4. **2.3 cannot be built** (see Open decisions).
5. **Gen-2 alternative counting:** see note b. The off-by-one in `backward_topk` is a live trap when writing Gen-2 configs.
6. **Gen 2's hybrid_smooth weights are** $w = \sigma(-2\rho_{j^\star}/T_{sel})$, a temperature-controlled rational soft-sign, not Gen 1's $U(u)$. It does match "two-alternatives forward".
7. **3.1 "soft forward** n=1**" reads one hard cell scaled by a smooth score.** It is piecewise-continuous with measured jumps (note d), not continuous.
8. **The margin confidence function is LookupFFN's kernel** $(\sum_j |u_j|) \prod_j \sigma(2|u_j|)$, not $\min_j |u_j|$. The factor 2 is fixed.
9. **Minor:** `exp_g_0193` (3.1) sets `lut_learnable_temps`, `lut_forward_mode` and `lut_forward_confidence`, all of which are inert on LightMHL.

Remaining work

- **Free / already done:** 2.1 (`exp_n_0121`), 3.1 (`exp_g_0193`), 3.2 (`exp_g_0195`).
- **To conduct:** 2.2 (hybrid_smooth forward, all-cells backward) and 2.4 (hybrid_smooth forward, 2-cell backward). Neither exists yet: every hybrid_smooth run on record is $n = 6$ with tied embeddings on the old eval (`exp_n_0044` $H8/tph64$, `exp_g_0009` $H8/tph32$, `exp_g_0010` $H16/tph32$, `exp_g_0011` $H8/tph256$), and the only `backward_topk>0` run, `exp_n_0188`, is $H4/tph128$ but $n = 7$ with a `bounded_norm` gate on.
- **To implement and run:** 1.1, 1.2, 3.3, plus 2.3 if `FastMultiHeadLut` is patched to allow a hard forward with `backward_topk>0`.
- **Condition for the new Gen-2 runs:** fork `exp_n_0121` verbatim (learnable temperatures, seed 1, 16K, batch 48 via device batch 12 $\times$ 4 accumulation, fixed-eval trainer), changing *only* `lut_forward_mode` / `lut_backward_topk`, so that 2.1/2.2/2.4 differ in nothing else.
- **Gen-1 wiring:** `MultiHeadLut` with `anchor_candidates` of shape [$tph = 128$, $H = 4$, $d_{in} = 48$] over a 192-dim compressed code, instead of [64, 8, 48] over 384.
- **Seeds:** there are **zero seed replicates at this geometry** (every run is `random_seed` 1 / `lut_base_seed` 1000). One extra seed of `exp_g_0193` ($\approx$0.9 h) would give the paper a real LUT seed spread.
- **Reference anchors** (vanilla dense FFN, corrected eval): 1.1651 / 1.1618 at 16K (two seeds), 1.1154 at 48K.

Appendix — prior evidence on confidence forms

Summarised from `experiments/ffn_replacement/runs_corrected/LOOKUPFFN_LINE.md` and `diag_confidence_forms.py`, which measure 6,291,456 real nap-8 margin vectors (`dump_margins.py`, model `sweep_s05_dout48_H4_tph256_c256_din32` at init). This partly answers note a's "other uncertainty functions". The regenerated cache reproduces the published numbers exactly (`min_margin_scale.py`).

Scale and within-token selectivity at init. The within-token spread is the part of the score that actually reweights the table ensemble; the across-token part only rescales a token's whole FFN output.

form	mean	p75/p25	within-token CV	reading
`bounded`	0.0542	2.06	0.54	18.4× forward attenuation at $n = 8$
`margin`	0.2286	3.04	0.87	self-normalises in training (0.94 after 4K steps)
`bounded_norm`	0.6838	1.09	0.061	nearly constant per token, so the linear `decompress` absorbs it
`min_margin` (proposed)	0.0059	7.32	1.78	39× below `margin`; needs $g \approx 38$ (note e)

4K-step confidence-gate arms vs the gate-off baseline S5 at **1.434572**: `bounded` unmatched +0.225 (stopped at 3,200 steps, widening); `bounded` × 12.61 +0.0004; `bounded_norm` +0.0066 (inside the 4K proxy's three-seed sd of 0.0096, which is a **lower bound** because `random_seed` re-draws only 26.8% of the parameters); `bounded_norm` on the Light implementation +0.0431; `margin` × 2.99 −0.0021. **Correction to how these gains are usually quoted:** the × 12.61 and × 2.99 arms were scale-matched to **`bounded_norm`'s mean 0.6838**, not to gate-off's 1.0. So scale, not the form, broke `bounded`, and `margin` was the only gate to land below the baseline.

16K, LightMHL: the clean pair `exp_g_0189` (`bounded_norm`) → `exp_g_0193` (`margin`) differs in nothing but the form and moves bpb by **−0.0346** (10.3× the 0.00335 vanilla two-seed range at 16K, which is not an LUT seed spread; see Protocol). This is why `margin` became the standard.